Mini SIEM Threat Detection Platform – Design Document Specification

Design Document Specification (DDS)
Mini SIEM Threat Detection Platform
Version: 1.0
Author: Lead System Architect

PART A — HIGH-LEVEL DESIGN (HLD)

1. System Architecture Overview

Architecture Style:
The system follows a Microservices and Event■Driven Architecture.
This allows high throughput log ingestion, independent service scaling,
fault tolerance, and asynchronous processing which is ideal for SIEM workloads.

C4 Architecture Levels:
Context Level – External actors interact with the SIEM platform.
Container Level – Web UI, API, Log Collector, Message Broker, Detection Engine, Storage.
Component Level – Internal modules such as rule engine, enrichment services, alert manager.
Code Level – Classes and functions implementing each service.

System Context:
External Systems → Mini SIEM Platform → Security Operators
Servers, Applications, and Network Devices send logs to the platform.
SOC Analysts and Administrators interact through dashboards.

Container Architecture:
Agents → Log Collector → Kafka → Detection Engine → Storage → Dashboard

Major Containers:
Web Application – Security monitoring dashboard
Backend API – REST services and authentication
Log Collector – High throughput ingestion service
Message Broker – Kafka cluster
Detection Engine – Event processing and rule evaluation
Database Layer – Elasticsearch, PostgreSQL, Redis
Agent – Lightweight log shipper deployed on servers

Technology Stack Justification:
Backend API – FastAPI (async, high performance)
Message Queue – Apache Kafka (durable, scalable)
Search Engine – Elasticsearch (log analytics and search)
Stream Processing – Kafka Streams / Faust
Frontend – React with visualization libraries
Containerization – Docker with Kubernetes orchestration
Monitoring – Prometheus and Grafana

2. Data Architecture

Data Flow:
Agents → Log Collector API → Kafka → Detection Engine → Elasticsearch → Dashboard
↓ ↓ ↓
Validation Rule Engine Alerts

Raw Log Schema:

```
{
 "id": "uuid",
 "timestamp": "datetime",
 "raw_message": "string",
 "source_type": "string",
 "host": "string",
 "ingestion_time": "datetime"
}
```

Normalized Event Schema:

```
{
 "event_id": "uuid",
 "timestamp": "datetime",
 "source_ip": "ip",
 "destination_ip": "ip",
```

```
 "username": "string",
 "event_type": "string",
 "severity": "string",
 "host": "string",
 "service": "string"
}
```

Storage Strategy:
Hot Storage – Elasticsearch SSD (30 days)
Warm Storage – Elasticsearch HDD (30■90 days)
Cold Storage – Object storage archive (90+ days)
Metadata – PostgreSQL
Cache – Redis

Data Retention:
Index Lifecycle Management moves data through Hot → Warm → Cold → Delete phases.

3. Integration Architecture

API Design:
RESTful APIs with JWT authentication and API keys.
Rate limiting applied to ingestion endpoints.
Versioned APIs using /api/v1 paths.

External Integrations:
Threat Intelligence – AlienVault OTX, VirusTotal
Notifications – Slack, Email SMTP, PagerDuty
Authentication – LDAP, SAML, OAuth2
Log Sources – Syslog, Windows Event Collector, cloud logs

4. Security Architecture

Defense in Depth:
Network segmentation
API gateway security
Mutual TLS between services
Database encryption

Authentication:
JWT based authentication
RBAC permissions for analysts, engineers, and administrators.

Secrets Management:
Secrets stored in secure secret stores such as Vault or Kubernetes Secrets.
Encryption keys rotated regularly.

Audit Trail:
Audit logs track authentication events, rule changes, alert updates,
and administrative actions.

5. Scalability and Performance

Horizontal Scaling:
Stateless services scale independently.
Kafka partitions allow parallel consumers.
Elasticsearch clusters scale with additional nodes.

Performance Targets:
Log ingestion ≥ 5000 events per second.
Search queries < 2 seconds.
Alert generation latency < 5 seconds.

6. Deployment Architecture

Container Orchestration:
Kubernetes manages containers, scaling, and deployment.

Environment Strategy:
Development – local environment
Staging – integration testing
Production – highly available cluster
Disaster Recovery – secondary region replication

Infrastructure as Code:
Terraform and Kubernetes manifests manage infrastructure.

Network Architecture:

VPC with segmented subnets for ingress, application services, and databases.

PART B — LOW■LEVEL DESIGN (LLD)

7. Component Level Design

Log Collector Service:
Receives logs from agents, validates schemas, enriches metadata,
and publishes events to Kafka topics.

Example Class Structure:

```
class LogCollector:
    endpoint: str
    buffer_size: int

    def receive_log(self, raw_log):
        pass

    def validate_schema(self, log):
        pass

    def enrich_with_metadata(self, log):
        pass

    def publish_to_kafka(self, log):
        pass
```

Sequence Flow:
Agent → Log Collector API → Validation → Enrichment → Kafka → Acknowledgment

API Endpoint:
POST /api/v1/logs
Headers: X-API-Key
Response: 202 Accepted

Detection Engine:

```
class Rule:
    id: str
    name: str
    severity: str

class RuleEngine:
    rules: list

    def process_event_stream(self):
        pass

    def generate_alert(self):
        pass
```

Rule Evaluation:
Each event is checked against active rules.
If a condition matches, an alert is generated.

Alert Manager:
Alert Lifecycle:
Detection → Deduplication → Enrichment → Routing → Notification → Case Creation

Dashboard API:
GET /api/v1/dashboard/summary
Returns event statistics and alert counts.

Database Schema (simplified):

users table – authentication and RBAC
rules table – detection rule definitions
alerts table – generated security alerts
alert_comments – case management notes

Elasticsearch Mapping:

```
{
 "@timestamp": "date",
 "source.ip": "ip",
 "event.type": "keyword",
 "message": "text"
```

```
}
```

8. API Contracts

Example Log Ingestion Request:

```
curl -X POST /api/v1/logs

Body:
{
 "message": "failed login",
 "host": "server1"
}
```

Response:
202 Accepted

9. Component Interactions

Log Processing Pipeline:
Agent → Log Collector → Kafka → Detection Engine → Elasticsearch
→ Alert Manager → Dashboard

Alert Investigation Flow:
Analyst opens dashboard → reviews alerts → investigates events
→ adds comments → resolves incident.

10. Error Handling

Kafka Down – Logs buffered and retried.
Elasticsearch Down – Events queued until recovery.
Detection Engine Crash – Kafka consumer rebalance.

Retry Strategy:
Exponential backoff and dead letter queues.

11. Monitoring and Observability

Metrics:
Events per second
Kafka consumer lag
Alert generation rate
API latency

Logging:
Structured JSON logs with correlation IDs.

Alerting:
System health alerts triggered when thresholds exceeded.

12. Development Guidelines

Coding Standards:
Python PEP8 with type hints
JavaScript linting rules

Testing:
Unit tests, integration tests, performance tests.

CI/CD Pipeline:
Commit → Build → Test → Security Scan → Deploy.

13. Appendices

Glossary:
SIEM – Security Information and Event Management
SOC – Security Operations Center
EPS – Events Per Second
RBAC – Role Based Access Control

References:
MITRE ATT&CK Framework
OWASP Security Guidelines
Elastic Common Schema